

Ranking candidates the way a great recruiter actually would

A hybrid lexical-semantic + structured-feature ranker for the Senior AI Engineer JD, built on top of a 100K-candidate pool that was profiled in depth before a single line of scoring logic was written.

100,000

candidates scanned

21

honeypots fingerprinted

~90s

full-pool runtime

0

network calls at ranking time

This JD tells you, in writing, what NOT to do

“The ‘right answer’ to this JD is not ‘find candidates whose skills section contains the most AI keywords.’ That’s a trap we’ve explicitly built into the dataset.”

“A Tier 5 candidate may not use the words ‘RAG’ or ‘Pinecone’ in their profile, but if their career history shows they built a recommendation system at a product company, they’re a fit.”

“A perfect-on-paper candidate who hasn’t logged in for 6 months and has a 5% recruiter response rate is, for hiring purposes, not actually available.”

— *job_description.docx*, “Final note for participants”

WHAT THIS MEANS FOR THE SYSTEM

-  Keyword/embedding similarity is one input, never the whole answer.
-  Career narrative content matters as much as the skills list.
-  Behavioral signals can discount a great fit — never rescue a bad one.
-  Traps (honeypots, keyword-stuffers) need to be found, not assumed away.

We profiled all 100,000 candidates before writing any scoring code



Only 47 titles exist

100K candidates use just 47 distinct job titles. The most senior AI/ML titles the JD describes total 29 candidates.



Honeypots have a clean fingerprint

“expert” proficiency + 0 months used. Bimodal across the pool: 0, or 3–5 — never 1 or 2. Exactly 21 candidates match.



Real FAANG names are rare anchors

Meta, Google, Netflix, Amazon, Microsoft, Apple appear 1–7 times each (43 total) vs ~7,500 for generic placeholders.



Behavioral twins are real

Thousands of candidates share identical title+company+experience+location, differing only in platform activity signals.



Consulting-only is a clean field

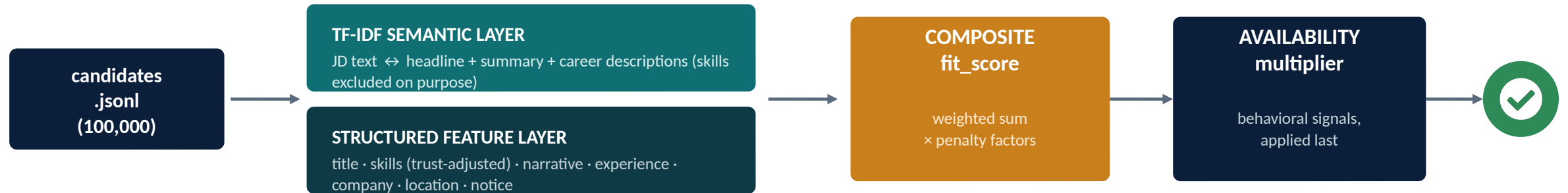
Every TCS/Infosys/Wipro/Accenture/Cognizant candidate carries `current_industry ∈ {IT Services, Consulting}` — directly matches the JD's disqualifier.



Generic English words are a trap

“production”, “evaluate”, “scale” false-fire on 20–46K unrelated profiles. Every narrative regex we kept was validated to ~0% false-positive rate first.

Hybrid scoring: two independent signal layers, then behavior



Honeypot check runs first and short-circuits directly to a near-zero score, bypassing the rest of the pipeline.

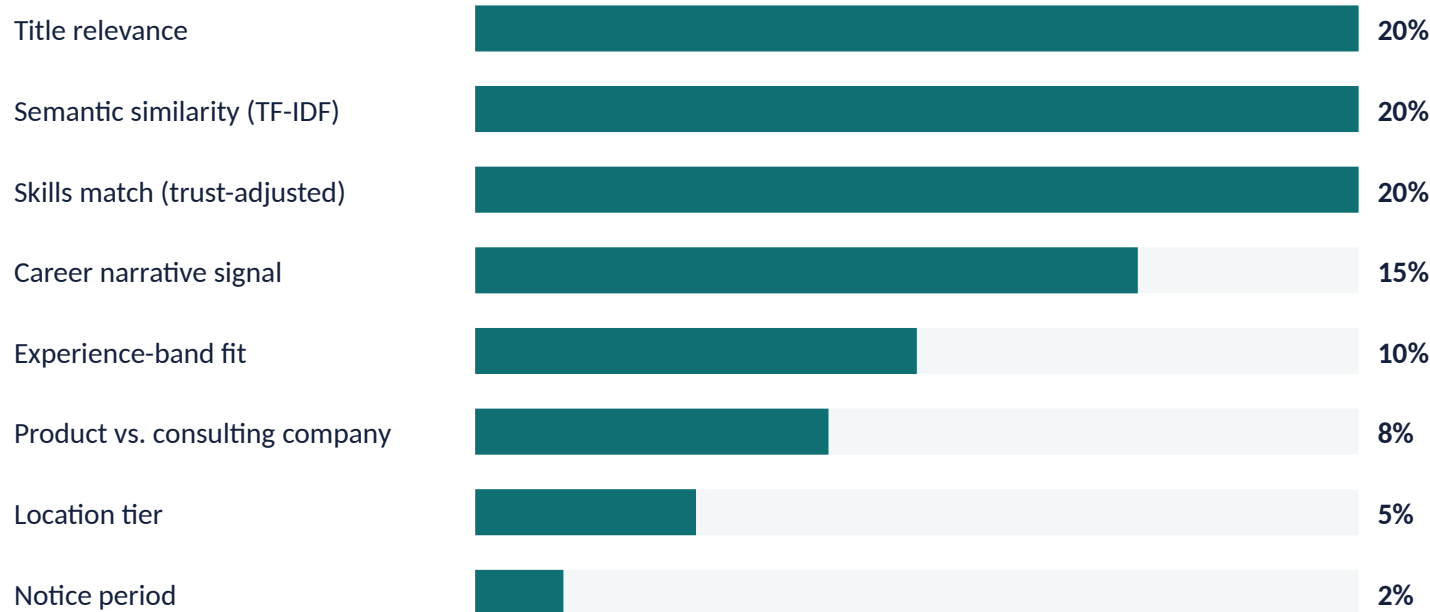
WHY THIS SHAPE, NOT AN LLM CALL PER CANDIDATE OR A NEURAL EMBEDDING MODEL

- Spec rules out hosted LLM calls and GPUs during ranking, and warns that 100K LLM calls won't fit 5 minutes even running locally.
- TF-IDF + cosine over the full 100K pool benchmarks at <30s on one CPU core — near-zero Stage-3 reproduction risk vs. shipping model weights.
- Skills are excluded from the similarity text on purpose, so keyword-stuffing in a profile can't inflate semantic score on top of the separate skill-match feature.

THE FORMULA

Eight weighted signals, six penalty checks, one final discount

fit_score — weighted components



× penalty factors (multiplicative)

- ⚠️ Keyword-stuffing (flashy tags, little substance)
- ⚠️ CV/speech-only without NLP/IR exposure
- ⚠️ Title-chasing (very short avg. tenure)
- ⚠️ No recent hands-on code (architecture/manager titles)
- ⚠️ Self-reported skill vs. platform-assessment mismatch
- ⚠️ Years-of-experience vs. career-history inconsistency

final_score = fit_score × availability_multiplier

Behavioral signals can discount a great fit — never rescue a poor one.

Four traps, four validated, falsifiable detection rules



Honeypots

proficiency = "expert" AND duration_months = 0

21 of 100,000 candidates. Bimodal (0, or 3-5 — never 1-2). Hard-suppressed before scoring; 0 leaked into our top 100.



Keyword stuffers

≥5 flashy surface skills, avg. duration <12mo, <25% substantive-skill ratio

Multiplicative penalty (×0.5). Validated on sample_candidates.json's CAND_0000001 — 17 AI tags, pure data-engineering career.



Behavioral twins

identical (title, company, experience, location) — redrob_signals breaks the tie

5,028 groups dataset-wide. Confirmed within the AI/ML population: CAND_0009200 vs CAND_0024878, same role, 0.32 vs 0.84 response rate.



Consulting-only careers

career-history industries ⊆ {IT Services, Consulting}, no product-company exposure

21,410 consulting-firm candidates DO have prior product history — exception path is real, not theoretical, per the JD's own carve-out.

RESULTS

What actually comes out the other end

RANK	SCORE	CANDIDATE	WHY
#1	0.699	Lead AI Engineer @ Razorpay	Information Retrieval, Learning to Rank, Vector Search
#2	0.662	Staff ML Engineer @ Yellow.ai	Learning to Rank, BM25, Elasticsearch
#6	0.638	Senior Applied Scientist @ Meta	16.2 yrs — flagged above the 5–9yr band
#21	0.581	Recommendation Systems Eng. @ Swiggy	Pinecone, Embeddings, Information Retrieval
#100	0.457	Search Engineer @ PharmEasy	Toronto, Canada — outside India, no visa sponsorship

0.699 → 0.457

score range across the top 100

0 / 100

honeypots in the final list

0 / 100

consulting-only-career leaks

0

exact-duplicate reasoning strings

Measured, not estimated, against every Stage-3 constraint

	Wall-clock time	≤ 5 minutes	~90 seconds	
	Memory	≤ 16 GB	~2.9 GB peak	
	Compute	CPU only, no GPU	CPU only — scikit-learn TF-IDF + pure Python	
	Network	off during ranking	zero network calls	
	Disk	≤ 5 GB intermediate state	no intermediate artifacts written	

Measured on a single CPU core with ~4GB RAM available — more constrained than the competition's stated 16GB/CPU-only target.

Single reproduction command:

```
python src/rank.py --candidates ./data/candidates.jsonl --out ./submission.csv
```


HONEST LIMITATIONS

What this system doesn't do, said plainly

- ! **TF-IDF isn't true paraphrase understanding.** “Improved retrieval relevance” and “made search results better” won't score as similar unless they share n-grams. Accepted tradeoff for this dataset's templated text, given the compute constraints — a production system without a 5-minute CPU budget would likely use a real embedding model.
- ! **Career-history text isn't always coupled to the listed title.** For “filler” candidates (Mechanical Engineer, Sales Executive, etc.) description text sometimes reads as a different job's description entirely. We found this directly via EDA and designed around it: the narrative signal is one of eight weighted inputs, not a dominant one, limiting the damage from any single mismatch.
- ! **Weights are reasoned, not learned.** No ground-truth relevance labels are available to participants, so the composite weights come from reading the JD plus validation against known example candidates — not from training a ranker on labeled data. Deliberately interpretable over deliberately optimal.
- ! **Semantic scores are pool-relative.** TF-IDF similarity is min-max normalized within this specific 100K-candidate run — meaningful for ranking, not as a portable absolute similarity metric across different candidate pools.

IN ONE SENTENCE

A recruiter judges fit, fairness, and reachability all at once — so the ranker scores all three, and lets none of them rescue the others.



Validated against the data, not assumed

Every rule traces to a query run against the full 100K pool (docs/data_analysis.md).



Zero honeypots, zero consulting-only leaks

Hard-suppressed and independently re-verified by qa_check.py.



~90s, ~2.9GB, single CPU core, no network

Comfortable margin under every Stage-3 constraint.



One command, bit-for-bit reproducible

`python src/rank.py --candidates ... --out ...` — verified twice, identical output.